МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 2

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «последовательный и параллельный алгоритмы умножения больших матриц»

Работу выполнил
Студент гр 4404
Завалин М.Г.

Казань, 2026

Цель работы:

Реализовать последовательный и параллельный (с помощью OpenMP) алгоритмы умножения больших матриц, измерить время их выполнения и сравнить эффективность при использовании 2, 4 и 8 потоков.

1. Задание

Задача 4. Напишите программу перемножения больших матриц. Сравните врем выполнения последовательной и параллельной программы на 2,4, 8 потоках (процессорах).

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком решающие задачу; каждый запуск, о котором говорится в задаче (разное число потоков, разные типы распределения, разные версии программы), подтверждается скриншотом с видимой командой запуска и полным выводом.

2. Ход работы

В ходе лабораторной работы была реализована программа для умножения больших матриц с последовательной и параллельной версиями на базе OpenMP: размеры матриц передавались через аргументы командной строки, а число потоков вводилось пользователем во время выполнения; память под матрицы выделялась через malloc и проверялась на успешность, после чего массивы инициализировались единицами, двойками и нулями; сначала выполнялось последовательное умножение с замером времени через omp_get_wtime, затем матрица C обнулялась, устанавливалось нужное число потоков через omp_set_num_threads и запускалось параллельное умножение по строкам с директивой #pragma omp parallel for и расписанием schedule(static); далее рассчитывалось ускорение как отношение времён, проверялось совпадение элемента C[0] в обеих версиях для подтверждения корректности.

Работа при использовании 2 потоков представлена на рисунке 1.

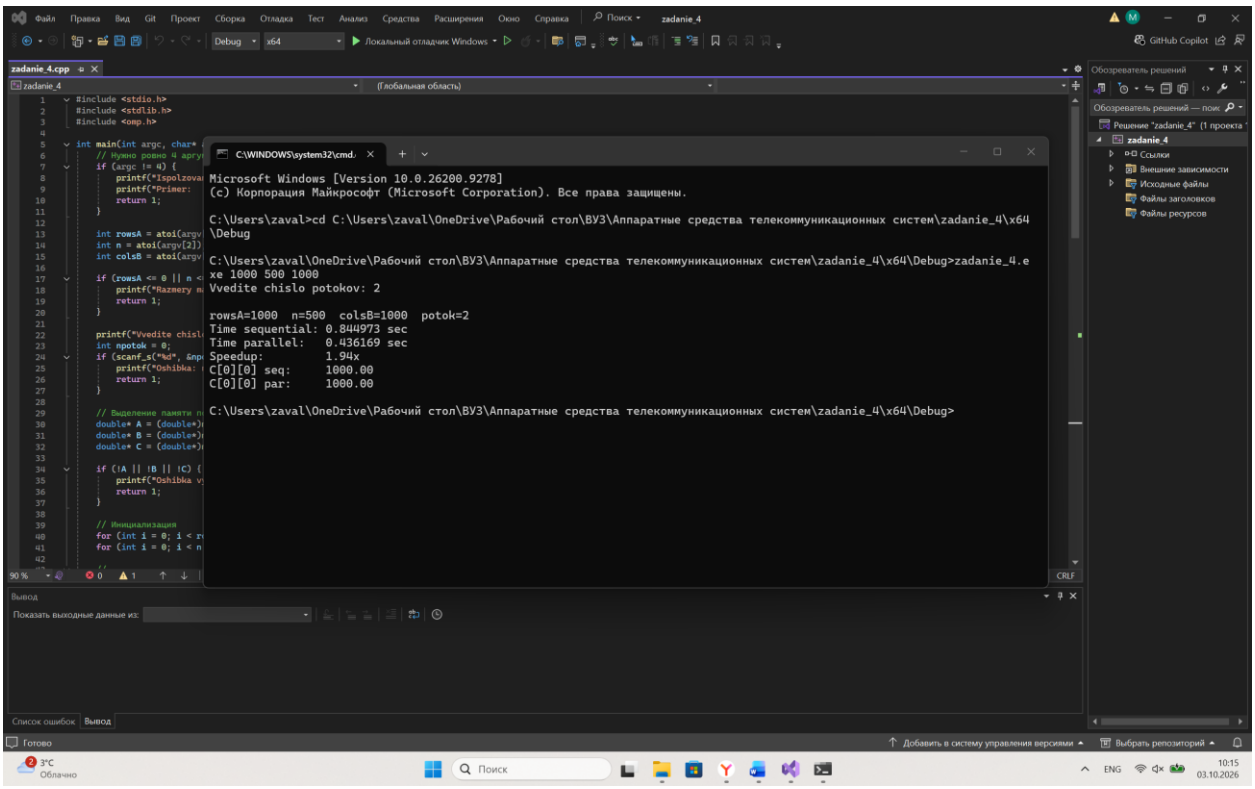


Рисунок 1 — запуск программы при 2 потоках

Исходя из результата при использовании 2 потоков, параллельное вычисление оказалось эффективнее последовательного почти в 2 раза.

Работа при использовании 4 потоков представлена на рисунке 2.

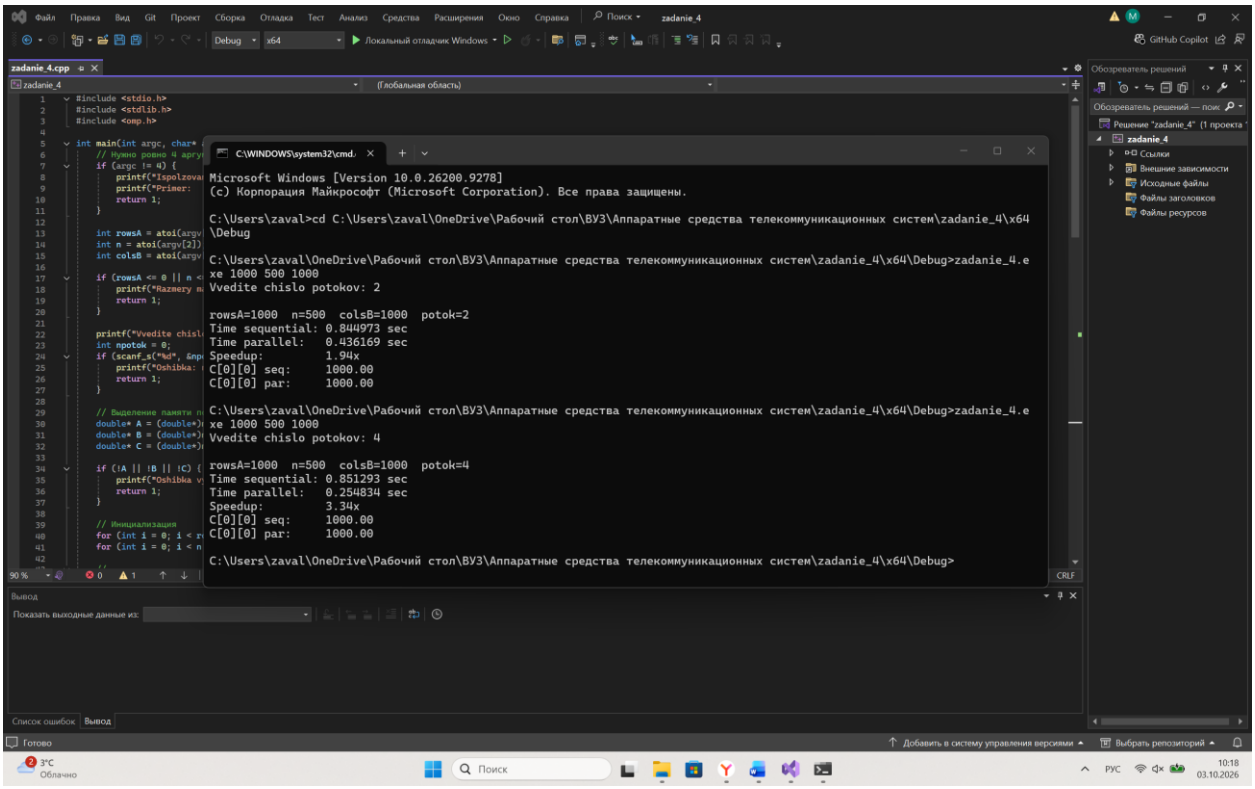


Рисунок 2 — запуск программы при 4 потоках

Исходя из результата при использовании 4 потоков, параллельное вычисление оказалось эффективнее последовательного почти в 3,5 раза.

Работа при использовании 8 потоков представлена на рисунке 3.

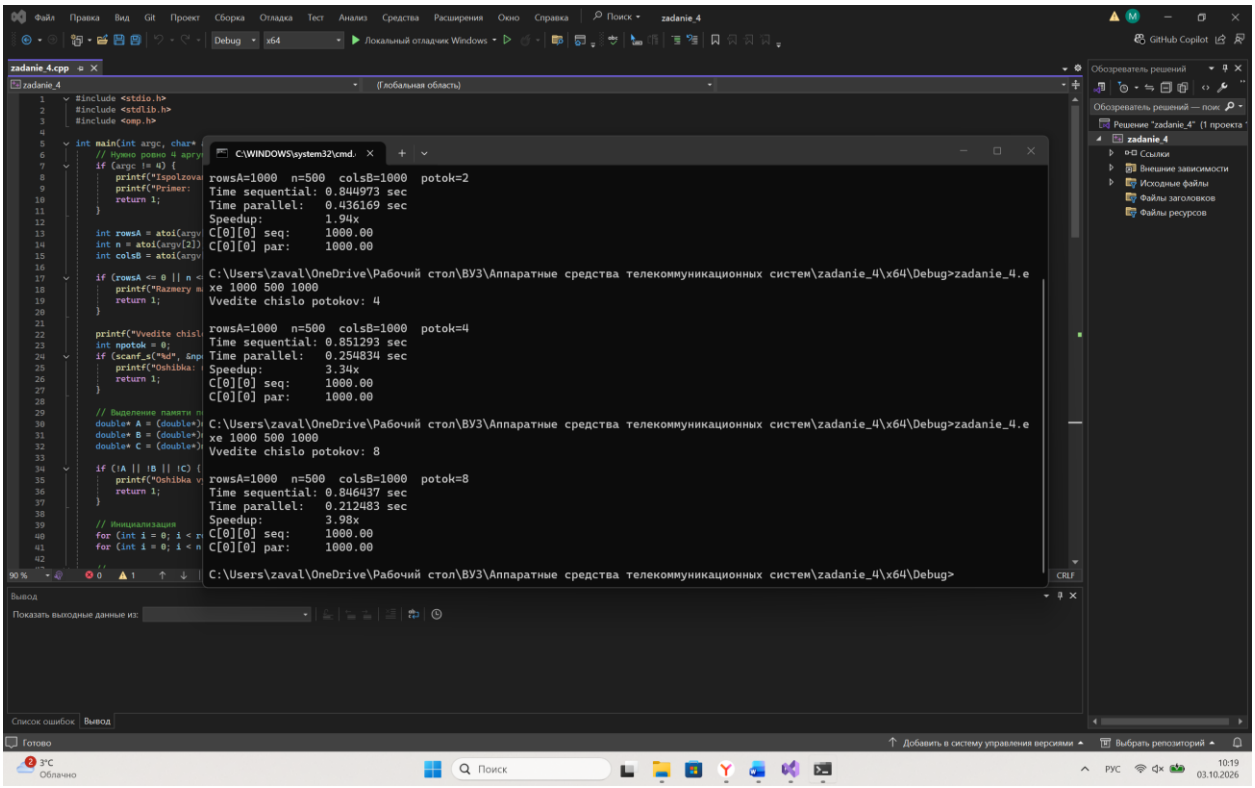


Рисунок 3 — запуск программы при 8 потоках

Исходя из результата при использовании 8 потоков, параллельное вычисление оказалось эффективнее последовательного почти в 4 раза.

Вывод: При увеличении потоков эффективность параллельного вычисления в сравнении с последовательным увеличивается.